These commands cover a broad range of activities from basic navigation and file manipulation to version control, container management, orchestration, database operations, and programming.

Definition

Cloud Infrastructure Management:

The process of designing, implementing, and maintaining scalable and reliable cloud-based infrastructure to support various applications and services. This involves using platforms like AWS, Azure, and Google Cloud to provide resources such as servers, storage, and networking.

Common Commands and Tasks

1. **Design and Implement Scalable Cloud Infrastructure:**

   - **AWS:** Use services like EC2, S3, RDS, VPC.

   - **Azure:** Utilize Azure Virtual Machines, Blob Storage, SQL Database, Virtual Network.

   - **Google Cloud:** Employ Compute Engine, Cloud Storage, Cloud SQL, VPC.

2. **Develop and Maintain CI/CD Pipelines:**

   - **Tools:** Jenkins, GitLab CI/CD, CircleCI, GitHub Actions.

   - **Commands:**

     ```bash

     # Example of Jenkinsfile for a basic CI/CD pipeline

     pipeline {

       agent any

       stages {

         stage('Build') {
     ```

```
            steps {
                sh 'make build'
            }
        }
        stage('Test') {
            steps {
                sh 'make test'
            }
        }
        stage('Deploy') {
            steps {
                sh 'make deploy'
            }
        }
    }
}
```

3. **Automate Infrastructure Provisioning and Configuration:**

   - **Terraform:**

   ```hcl
   resource "aws_instance" "example" {
     ami           = "ami-123456"
     instance_type = "t2.micro"
```

```
}
```

- **Ansible:**

```yaml
- name: Provision an EC2 instance
  hosts: localhost
  tasks:
    - name: Launch instance
      ec2:
        key_name: mykey
        instance_type: t2.micro
        image: ami-123456
        wait: yes
```

- **CloudFormation:**

```yaml
Resources:
  MyEC2Instance:
    Type: "AWS::EC2::Instance"
    Properties:
      InstanceType: t2.micro
      ImageId: "ami-123456"
```

4. **Monitor and Optimize Cloud Infrastructure:**

- **AWS CloudWatch:** Set up monitoring dashboards and alarms.
- **Azure Monitor:** Use metrics, logs, and alerts.
- **Google Cloud Monitoring:** Create custom dashboards and alerting policies.

5. **Collaborate with Development Teams:**

- **Tools:** Slack, Microsoft Teams, Jira, Confluence.
- **Practices:** Regular stand-ups, code reviews, shared documentation.

6. **Implement and Maintain Security Best Practices:**

- **IAM Policies:** Define and enforce access control.
- **Encryption:** Use encryption for data at rest and in transit.
- **Security Groups:** Configure firewall rules.
- **Regular Audits:** Perform security audits and compliance checks.

7. **Troubleshoot and Resolve Issues:**

- **Logs:** Analyze logs using CloudWatch, Azure Monitor, or Stackdriver.
- **Alerts:** Respond to alerts and incidents.
- **Debugging Tools:** Use cloud-native debugging tools and services.

8. **Develop and Maintain Documentation:**

- **Tools:** Markdown, Confluence, Google Docs.
- **Content:** Create detailed documentation for infrastructure setups, deployment processes, and troubleshooting steps.

Example Code Snippets

Terraform:

```hcl
provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "example" {
  bucket = "my-unique-bucket-name"
  acl    = "private"
}
```

Ansible Playbook:

```yaml
- hosts: localhost
  tasks:
    - name: Ensure Apache is installed
      yum:
        name: httpd
        state: present
    - name: Start Apache
      service:
```

```
    name: httpd
    state: started
```

CloudFormation Template:

```yaml
Resources:
  MyS3Bucket:
    Type: "AWS::S3::Bucket"
    Properties:
      BucketName: "my-unique-bucket-name"
```